UNG
Roland
20231933
L3ASR

Rapport de projet

Thème 1 : sauvegarde avec Borg

Projet administration système - ASR

Année universitaire 2025/2026

Table des matières

1. Sauvegardes avec Borgbackup 4
1.1. Adressages et interfaces 4
1.1.1. Interface de la station1 4
1.1.2. Interface du serveur2 4
1.2. Ecriture du script 5
1.3. Exécution du script 5
1.4. Différence lien physique et lien symbolique 6
1.4.1. Lien symbolique 6
1.4.2. Lien physique 6
1.5. Déduplication 6
1.6. Compression 7
1.7. Chiffrement des sauvegardes 7
1.7.1. Méthode `none` 7
1.7.2. Méthode `repokey` 7
1.7.3. Méthode `keyfile` 7
1.7.4. Méthode recommandée 7
1.8. Procédures de restauration de données 8
1.8.1. Initialisation Borg 8
1.8.2. Analyse des paquets 8
1.8.3. Restauration d'un dossier 9
1.8.4. Restauration d'un fichier 9
1.8.5. Restauration complète 9
2. Test de d'autres outils de sauvegardes 10
2.1. Outil Rsync 10
2.1.1. Test de sauvegardes 10
2.1.2. Vérification de la sauvegarde depuis serveur2 11
2.1.3. Analyse des paquets 11
2.2. Outil Rsnapshot 11
2.2.1. Configuration fichier 11
2.2.2. Test de configuration 12
2.2.3. Test de restauration 12

2.3. Outil Duplicati 12
2.3.1. Installation du paquet 12
2.3.2. Configuration dans l'interface 13
2.3.3. Restauration 16
2.4. Comparaison des outils de sauvegardes 17
2.4.1. Fonctionnalités 17
2.4.2. Restauration à l'identique 17
2.4.3. Performance 18
3. Sécurité des sauvegardes 18
3.1. Problématiques 18
3.1.1. Confidentialité 18
3.1.2. Intégrité 18
3.1.3. Disponibilité et ransomwares 19
3.2. Solutions des outils de sauvegardes 19
3.2.1. Chiffrement 19
3.2.2. Déduplication 19
3.2.3. Immuabilité 19

1. Sauvegardes avec Borgbackup

1.1. Adressages et interfaces

1.1.1. Interface de la station1

```
root@debian:~# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group defaul
t qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP gro
up default qlen 1000
    link/ether 00:0c:29:d5:dc:fc brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    inet 192.168.10.3/24 brd 192.168.10.255 scope global dynamic noprefixroute e
ns33
       valid_lft 1065sec preferred_lft 1065sec
    inet6 fe80::1023:36b3:d80b:d3f9/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
root@debian:~# ip route
default via 192.168.10.2 dev ens33 proto dhcp src 192.168.10.3 metric 100
192.168.10.0/24 dev ens33 proto kernel scope link src 192.168.10.3 metric 100
```

1.1.2. Interface du serveur2

```
root@debian:~# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group defaul
t qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP gro
up default qlen 1000
    link/ether 00:0c:29:31:14:4c brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    inet 192.168.10.4/24 brd 192.168.10.255 scope global dynamic noprefixroute e
ns33
       valid_lft 1080sec preferred_lft 1080sec
    inet6 fe80::95bd:2c05:45bc:29f5/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
root@debian:~# ip route
default via 192.168.10.2 dev ens33 proto dhcp src 192.168.10.4 metric 100
192.168.10.0/24 dev ens33 proto kernel scope link src 192.168.10.4 metric 100
```

1.2. Ecriture du script

Script à écrire dans cree-arbo.sh pour chaque utilisateur (cd /home/test1, cd /home/test2, cd /home/test3)

```
#!/bin/bash

USER=$(whoami)
BASE="$HOME/data-$USER"

mkdir -p "$BASE"
cd "$BASE" || exit 1

echo "Fichier de test pour $USER" > test-$USER.txt
echo "Fichier caché pour $USER" > .test-dot-$USER.txt

ln -s test-$USER.txt test-sl-$USER.txt
ln test-$USER.txt test-hl-$USER.txt
```

1.3. Exécution du script

Rendre le script exécutable : chmod +x cree-arbo.sh

Exécuter le script depuis chaque utilisateur :

- su – nameUser
- ./cree-arbo.sh

Vérifier le résultat :

```
test1@debian:~$ ls -la data-test1
total 20
drwxr-xr-x  2 test1 test1 4096 24 janv. 19:08 .
drwx------ 14 test1 test1 4096 24 janv. 19:08 ..
-rw-r--r--  1 test1 test1   26 24 janv. 19:08 .test-dot-test1.txt
-rw-r--r--  2 test1 test1   27 24 janv. 19:08 test-hl-test1.txt
lrwxrwxrwx  1 test1 test1   14 24 janv. 19:08 test-sl-test1.txt -> test-test1.txt
-rw-r--r--  2 test1 test1   27 24 janv. 19:08 test-test1.txt

test2@debian:~$ ls -la data-test2
total 20
drwxr-xr-x 2 test2 test2 4096 24 janv. 19:54 .
drwx------ 3 test2 test2 4096 24 janv. 19:54 ..
-rw-r--r-- 1 test2 test2   26 24 janv. 19:54 .test-dot-test2.txt
-rw-r--r-- 2 test2 test2   27 24 janv. 19:54 test-hl-test2.txt
lrwxrwxrwx 1 test2 test2   14 24 janv. 19:54 test-sl-test2.txt -> test-test2.txt
-rw-r--r-- 2 test2 test2   27 24 janv. 19:54 test-test2.txt
```

```
test3@debian:~$ ls -la data-test3
total 20
drwxr-xr-x 2 test3 test3 4096 24 janv. 19:56 .
drwx------ 3 test3 test3 4096 24 janv. 19:56 ..
-rw-r--r-- 1 test3 test3   26 24 janv. 19:56 .test-dot-test3.txt
-rw-r--r-- 2 test3 test3   27 24 janv. 19:56 test-hl-test3.txt
lrwxrwxrwx 1 test3 test3   14 24 janv. 19:56 test-sl-test3.txt -> test-test3.txt
-rw-r--r-- 2 test3 test3   27 24 janv. 19:56 test-test3.txt
```

1.4. Différence lien physique et lien symbolique

1.4.1. Lien symbolique

Un lien symbolique est un fichier spécial qui contient un chemin vers un autre fichier

Comme on le voit dans les trois captures précédentes, le fichier test-sl-testX.txt contient le chemin vers le fichier test-testX.txt

1.4.2. Lien physique

Lien physique : deux noms différents qui pointent vers le même contenu, ils ont le même inode (numéro interne).

```
test1@debian:~$ ls -li data-test1
total 8
914398 -rw-r--r-- 2 test1 test1 27 24 janv. 19:08 test-hl-test1.txt
914400 lrwxrwxrwx 1 test1 test1 14 24 janv. 19:08 test-sl-test1.txt -> test-test1.txt
914398 -rw-r--r-- 2 test1 test1 27 24 janv. 19:08 test-test1.txt
```

Comme on le voit dans la capture, il y a deux fichiers différents qui ont le même inode 914398.

C'est important pour les sauvegardes car certains outils de sauvegardes utilisent les liens physiques pour économiser de l'espace. Borg détecte que deux fichiers sont identiques

1.5. Déduplication

La déduplication consiste à ne stocker qu'une seule fois les données identiques. Borg découpe le fichier en bloc et ne stocke qu'une seule fois le bloc s'il n'existe pas encore dans le dépôt. Son intérêt est de réduire l'espace du disque et de faire des sauvegardes incrémentales automatiques.

Elle s'effectue avant la compression et avant le chiffrement. Elle s'applique sur les blocs de données, pas sur les fichiers entiers

1.6. Compression

La compression s'applique après la déduplication, sur les blocs uniques. Borg compresse la taille des blocs stockés pour réduire l'espace du disque. Comme les données sont compressés avant l'envoi, cela réduit le volume transféré sur le réseau.

Mode auto : Borg choisit automatiquement la meilleure méthode selon le type de données. Cela permet de ne pas perdre du temps à compresser ce qui ne peut pas l'être, gagner de l'espace quand c'est utile et rester rapide.

1.7. Chiffrement des sauvegardes

1.7.1. Méthode `none`

Les données sont stockées en clair dans le dépôt, sur le serveur de sauvegarde. N'importe qui ayant accès au dépôt pourra lire les fichiers. Tout est visible, il n'y a donc pas de confidentialité et n'est pas adapté s'il y a des données importantes.

1.7.2. Méthode `repokey`

Les données sont chiffrées du côté client, et la clé de chiffrement est stockée dans le dépôt Borg lui-même sur le serveur, protégée par une passphrase. Le seul inconvénient est si quelqu'un connait ou force le passphrase, il pourra déchiffrer toutes les données.

1.7.3. Méthode `keyfile`

Les données sont chiffrées du côté client, et la clé de chiffrement est stockée dans un fichier local sur la machine cliente, protégée par une passphrase. L'inconvénient est que si le fichier est perdu, alors la clé aussi, donc les sauvegardes ne seront plus jamais déchiffrées

1.7.4. Méthode recommandée

La meilleure méthode est `repokey` car tout d'abord, les données sont chiffrées, donc elles ne sont pas visibles par tout le monde. La clé de chiffrement est stockée dans un

dépôt, ce qui évite de gérer un fichier séparé. De plus cette clé est protégée par une passphrase. Cette méthode offre une bonne sécurité et est simple à utiliser.

1.8. Procédures de restauration de données

1.8.1. Initialisation Borg

Sur serveur2, création d'un dossier dédié pour la sauvegarde :

sudo mkdir -p /srv/borg/station1

sudo chown debian:debian /srv/borg/station1

Depuis station 1, initialisation borg :

```
root@debian:~# borg init --encryption=repokey-blake2 debian@192.168.10.4:/srv/borg/station1
debian@192.168.10.4's password:
Enter new passphrase:
Enter same passphrase again:
Do you want your passphrase to be displayed for verification? [yN]: y
Your passphrase (between double-quotes): "borg"
Make sure the passphrase displayed above is exactly what you wanted.

By default repositories initialized with this version will produce security
errors if written to with an older version (up to and including Borg 1.0.8).

If you want to use these older versions, you can disable the check by running:
borg upgrade --disable-tam ssh://debian@192.168.10.4/srv/borg/station1

See https://borgbackup.readthedocs.io/en/stable/changes.html#pre-1-0-9-manifest-spoofing-vulnerability
 for details about the security implications.

IMPORTANT: you will need both KEY AND PASSPHRASE to access this repo!
If you used a repokey mode, the key is stored in the repo, but you should back it up separately.
Use "borg key export" to export the key, optionally in printable format.
Write down the passphrase. Store both at safe place(s).
```

Création d'une archive et liste des archives :

```
root@debian:~# borg create debian@192.168.10.4:/srv/borg/station1::archive /home/test1/data-test1
debian@192.168.10.4's password:
Enter passphrase for key ssh://debian@192.168.10.4/srv/borg/station1:
root@debian:~# borg list debian@192.168.10.4:/srv/borg/station1
debian@192.168.10.4's password:
Enter passphrase for key ssh://debian@192.168.10.4/srv/borg/station1:
test-backup                          Sun, 2026-01-25 19:46:09 [13c1be9524dfb07eb1996525d224ab7183c9c44b7f45a04c7c9b1b7ba4282b29]
archive                              Sun, 2026-01-25 22:10:13 [4249c2d74ddaf81bbb1331ddb4ae38a43464d4b5c73dc9ea28f6fa1baa57594d]
```

1.8.2. Analyse des paquets

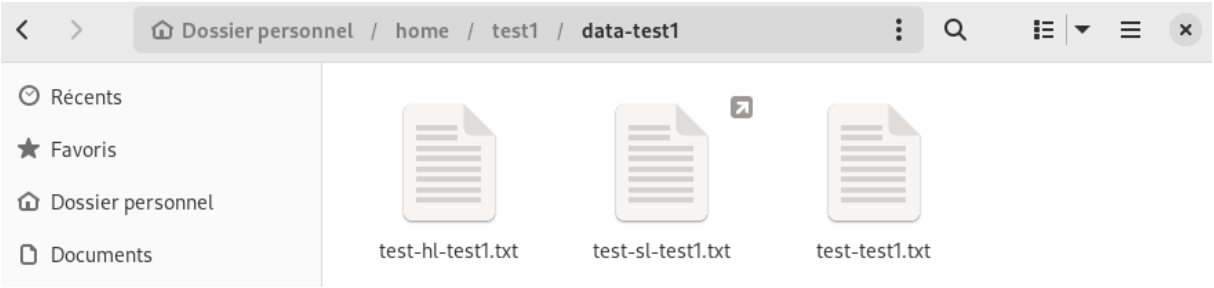
```
▾ SSH Protocol
    SSH Version 2 (encryption:chacha20-poly1305@openssh.com compression:none)
    [Direction: client-to-server]
```

D'après la capture de trame lors de la sauvegarde, on voit que la connexion SSH est chiffré : « encryption »

1.8.3. Restauration d'un dossier

Exemple pour le dossier data-test1

```
root@debian:~# borg extract debian@192.168.10.4:/srv/borg/station1::archive /home/test1/data-test1
debian@192.168.10.4's password:
Enter passphrase for key ssh://debian@192.168.10.4/srv/borg/station1:
```



1.8.4. Restauration d'un fichier

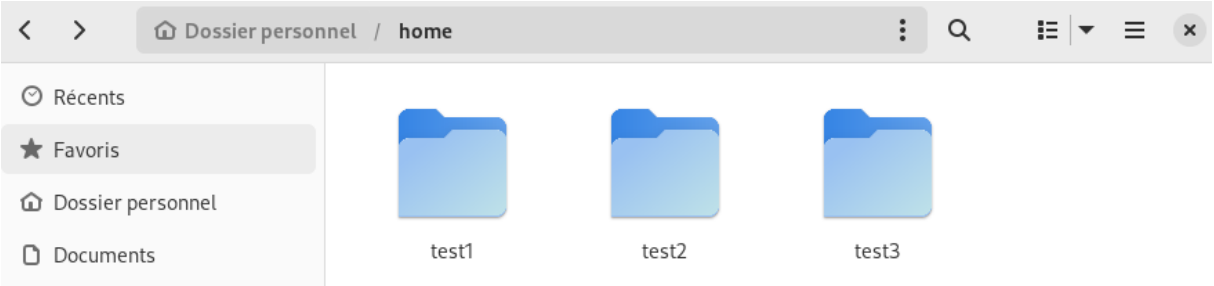
Exemple pour le fichier test-hl-test2.txt

```
root@debian:~# borg create debian@192.168.10.4:/srv/borg/station1::arc_test2 /home/test2/data-test2
debian@192.168.10.4's password:
Enter passphrase for key ssh://debian@192.168.10.4/srv/borg/station1:
root@debian:~# borg extract debian@192.168.10.4:/srv/borg/station1::arc_test2 /home/test2/data-test2/test-hl-test2.txt
debian@192.168.10.4's password:
Enter passphrase for key ssh://debian@192.168.10.4/srv/borg/station1:
```

1.8.5. Restauration complète

Exemple pour l'archive qui contient les dossiers test2 et test3

```
root@debian:~# borg create debian@192.168.10.4:/srv/borg/station1::gros_archive \
> /home/test2/data-test2 \
> /home/test3/data-test3
debian@192.168.10.4's password:
Enter passphrase for key ssh://debian@192.168.10.4/srv/borg/station1:
root@debian:~# borg extract debian@192.168.10.4:/srv/borg/station1::gros_archive
debian@192.168.10.4's password:
Enter passphrase for key ssh://debian@192.168.10.4/srv/borg/station1:
```



2. Test de d'autres outils de sauvegardes

On va sauvegarder : /home/test1/data-test1

Et stocker les sauvegardes dans : /backup/

On crée d'abord le dossier (dans la machine serveur2) pour chaque sauvegarde

`sudo mkdir -p /backup/rsync/test1` puis faire de même pour rsnapshot, restic ext...

`sudo chown debian:debian /backup/rsync/test1` faire de même avec les autres

2.1. Outil Rsync

2.1.1. Test de sauvegardes

Après l'avoir installé (depuis station1) :

```
root@debian:~# rsync -avz /home/test1/data-test1/ debian@192.168.10.4:/backup/rsync/test1/
debian@192.168.10.4's password:
sending incremental file list
./
.test-dot-test1.txt
test-hl-test1.txt
test-sl-test1.txt -> test-test1.txt
test-test1.txt

sent 367 bytes  received 79 bytes  99,11 bytes/sec
total size is 94  speedup is 0,21
```

2.1.2. Vérification de la sauvegarde depuis serveur2

```
root@debian:~# ls -l /backup/rsync/test1
total 8
-rw-r--r-- 1 debian debian 27 24 janv. 19:08 test-hl-test1.txt
lrwxrwxrwx 1 debian debian 14 24 janv. 19:08 test-sl-test1.txt -> test-test1.txt
-rw-r--r-- 1 debian debian 27 24 janv. 19:08 test-test1.txt
```

2.1.3. Analyse des paquets

```
▾ SSH Protocol
    SSH Version 2 (encryption:chacha20-poly1305@openssh.com compression:none)
    [Direction: client-to-server]
```

Les captures réseau de BorgBackup et rsync apparaissent identiques dans Wireshark, car les deux utilisent le protocole SSH pour le transfert. Les données étant chiffrées, on ne peut pas distinguer le contenu ou l'outil utilisé. Les paquets apparaissent sous forme de "Encrypted packet", avec des tailles similaires.

2.2. <u>Outil Rsnapshot</u>

2.2.1. Configuration fichier

Après avoir installé, on édite la configuration du fichier : sudo nano /etc/rsnapshot.conf

snapshot_root /backup/rsnapshot/test1/ #dossier de sauvegarde

retain daily 7 # 7 jours de sauvegarde

cmd_ssh /usr/bin/ssh

backup debian@192.168.10.3:/home/test1/data-test1/ 192.168.10.3/

On remplace ou met en commentaire toutes les autres lignes avec snapshot_root, retain et backup, sauf pour cmd. On utilise que des tabulations, il n'y a aucun espace

2.2.2. Test de configuration

```
root@debian:~# sudo rsnapshot configtest
Syntax OK
```

On lance une sauvegarde avec la commande : rsnapshot daily

Puis on vérifie :

```
root@debian:~# ls -l /backup/rsnapshot/test1
total 4
drwxr-xr-x 2 root root 4096 18 févr. 14:15 daily.0
```

Si on refait une sauvegarde

```
root@debian:~# ls -l /backup/rsnapshot/test1
total 8
drwxr-xr-x 2 root root 4096 18 févr. 14:21 daily.0
drwxr-xr-x 2 root root 4096 18 févr. 14:15 daily.1
```

L'outil renomme daily.0 en daily.1

Il crée un nouveau daily.0 en utilisant des liens physiques vers daily.1 pour ne copier que ce qui a changé

Daily.0 (plus récent) et daily.1 (plus ancien)

2.2.3. Test de restauration

```
root@debian:~# restic -r /sauvegardes/restic restore latest --target /tmp/restic-restore
repository 27cbeefe opened (repository version 2) successfully, password is correct
restoring <Snapshot 75bfe324 of [/home/test1/data-test1] at 2026-01-27 10:59:34.112208902
+0100 CET by root@debian> to /tmp/restic-restore
```

2.3. <u>Outil Duplicati</u>

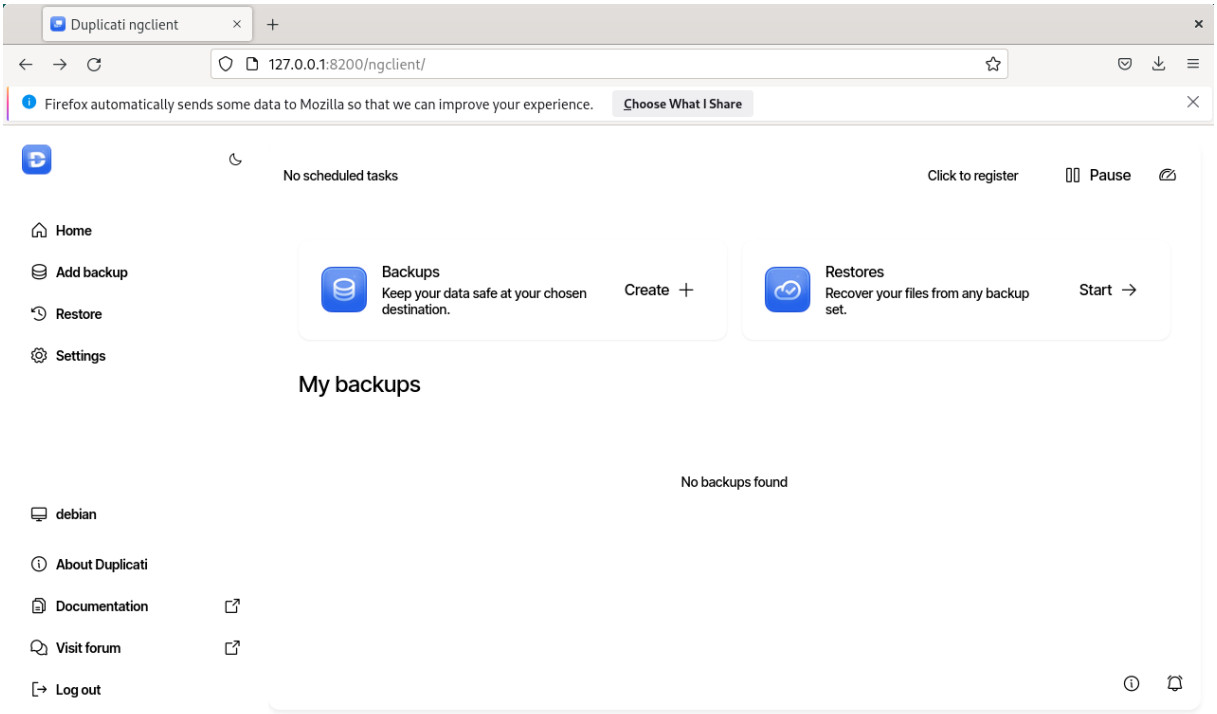
2.3.1. Installation du paquet

Télécharger le paquet depuis : https://www.duplicati.com/download

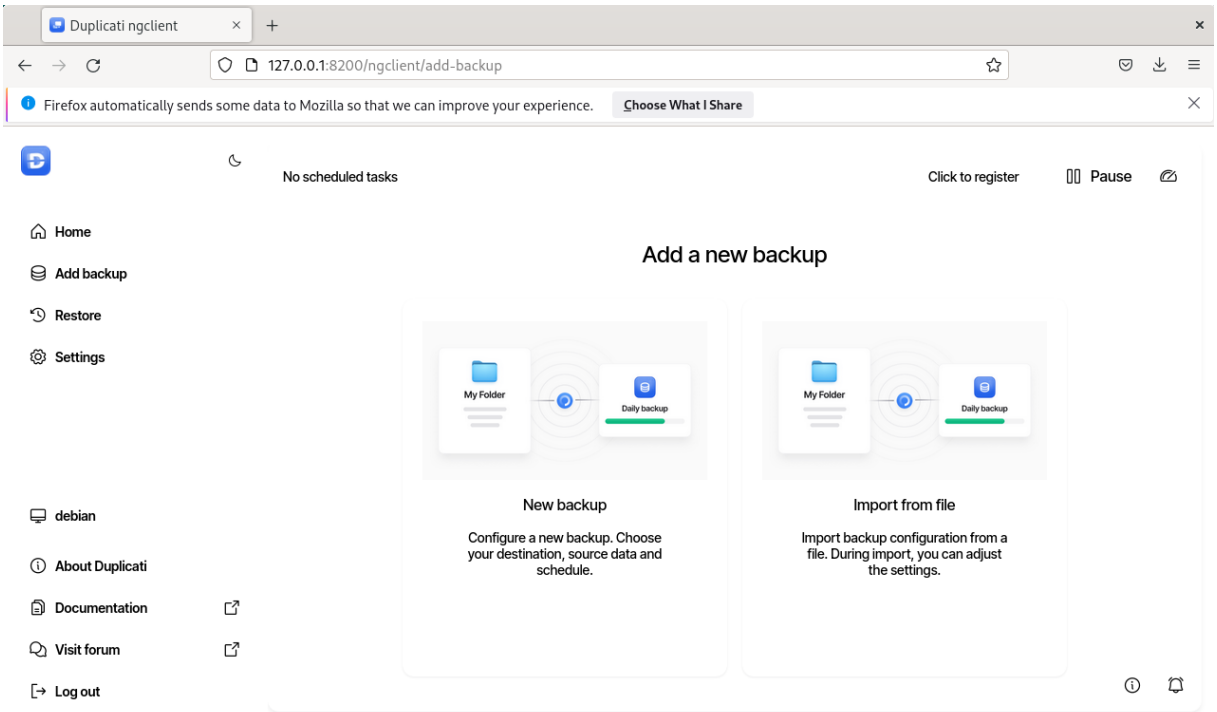
Puis installer : sudo dpkg -i nomFichier.deb

2.3.2. Configuration dans l'interface

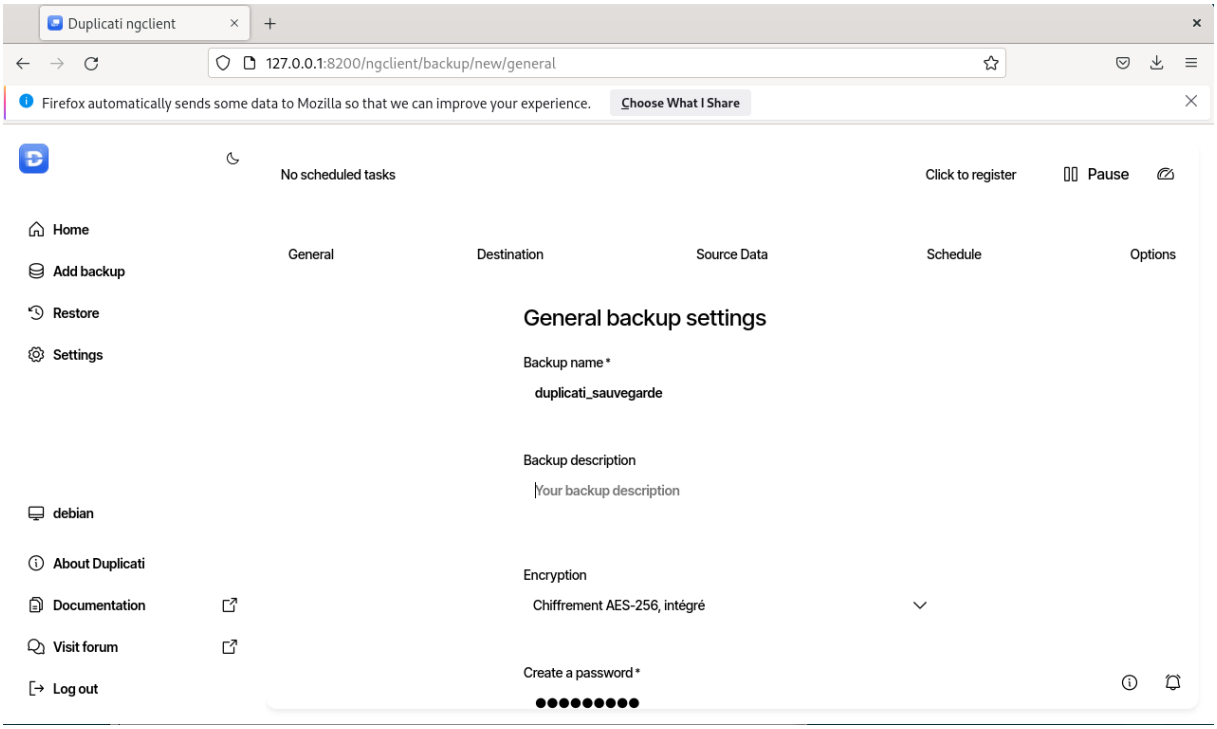
Avant de lancer l'interface, il faut taper la commande `duplicati`



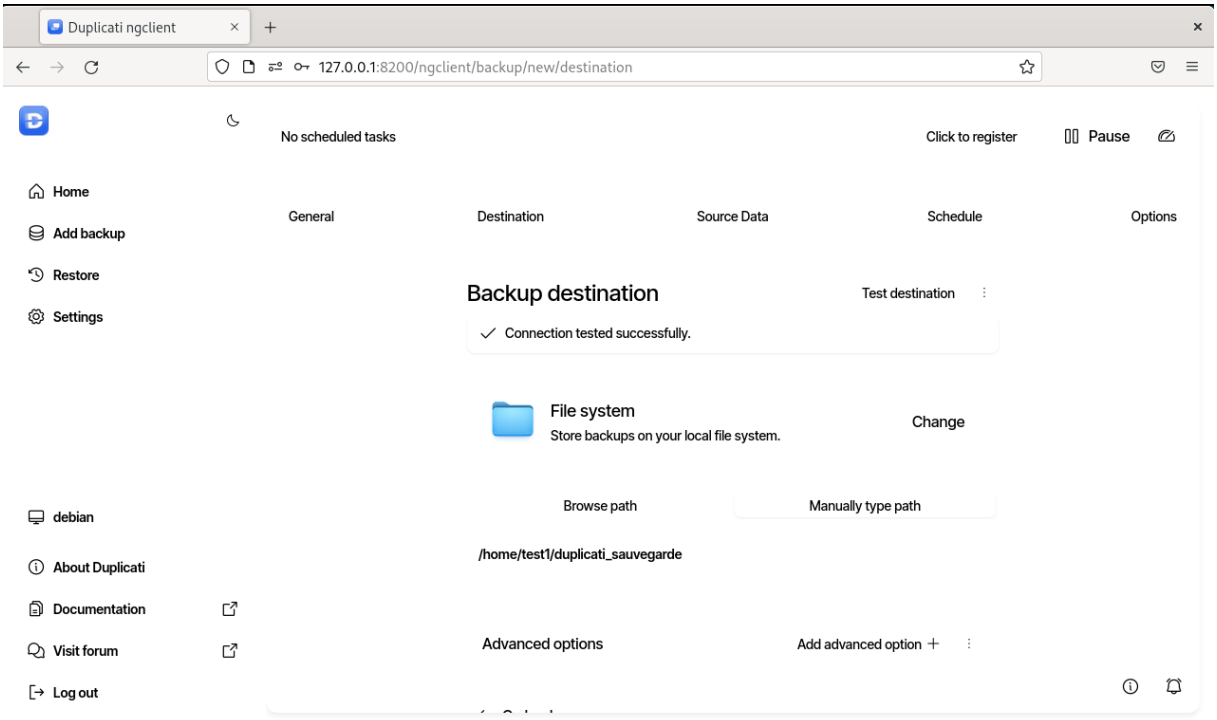
Cliquer Add backup



Cliquer New backup

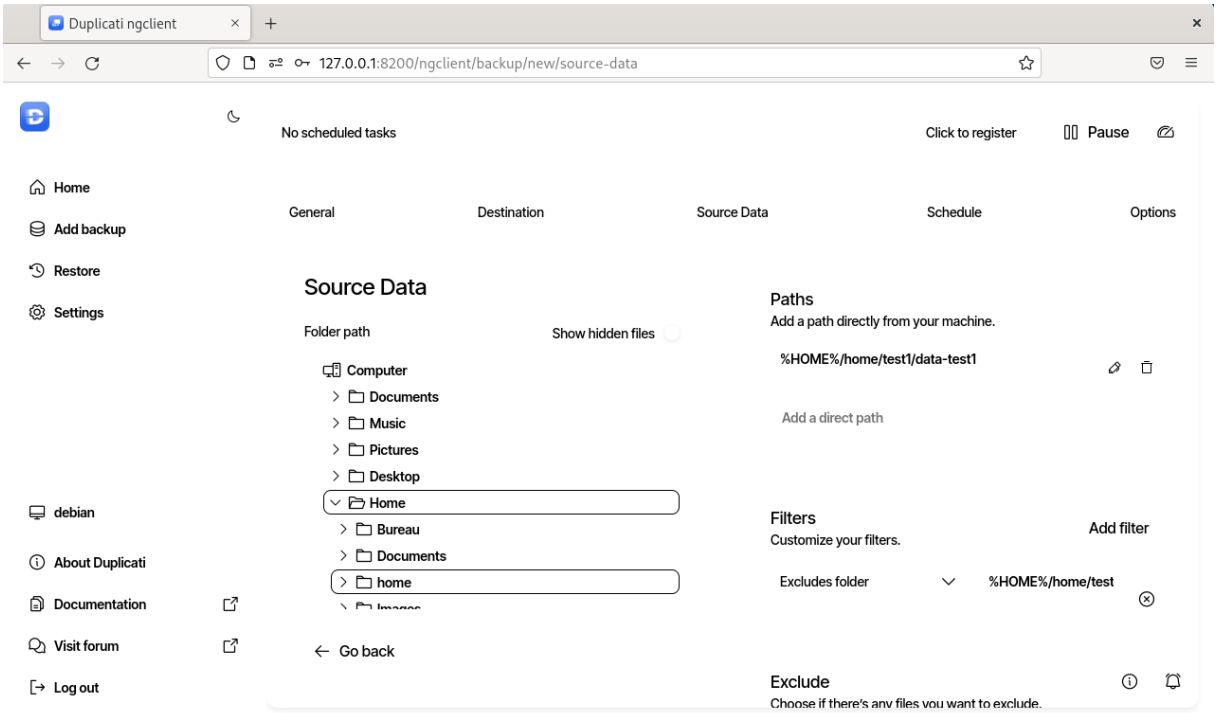


Rentrer le nom de la sauvegarde, le chiffrement et créer le mot de passe (duplicati)

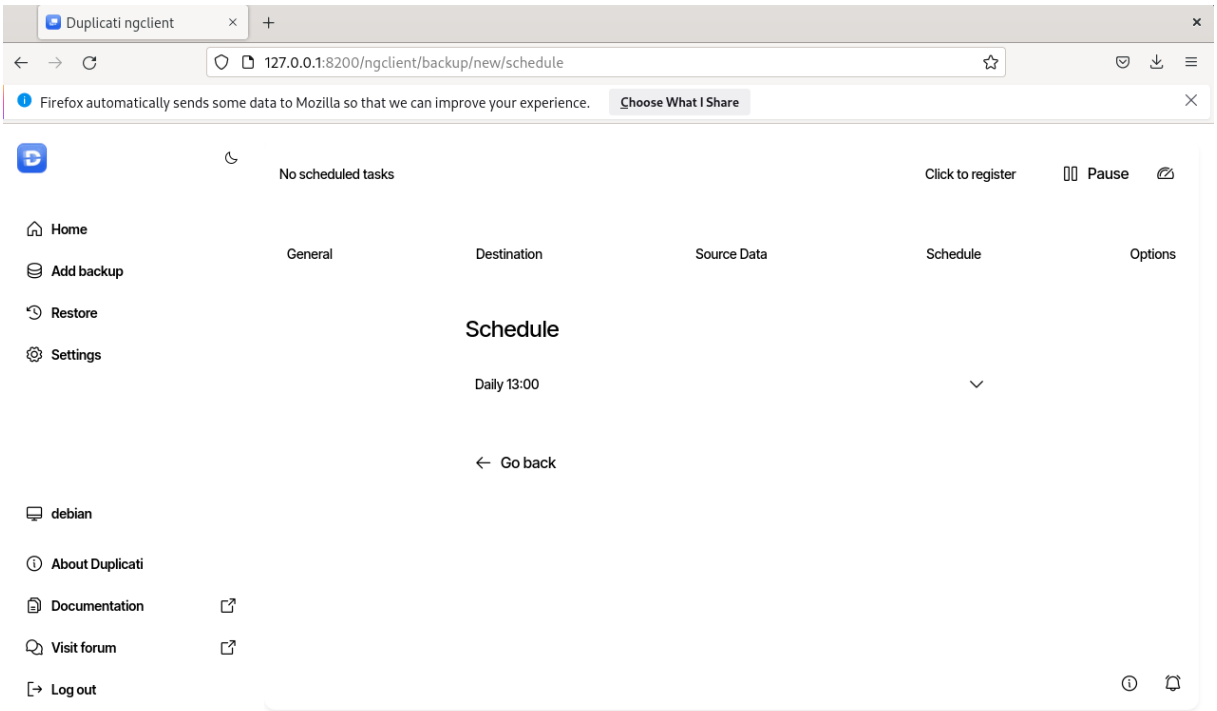


- Rentrer l'hôte du serveur (où on veut sauvegarder) : 192.168.10.4
- Le port 22
- Le nom d'utilisateur test1

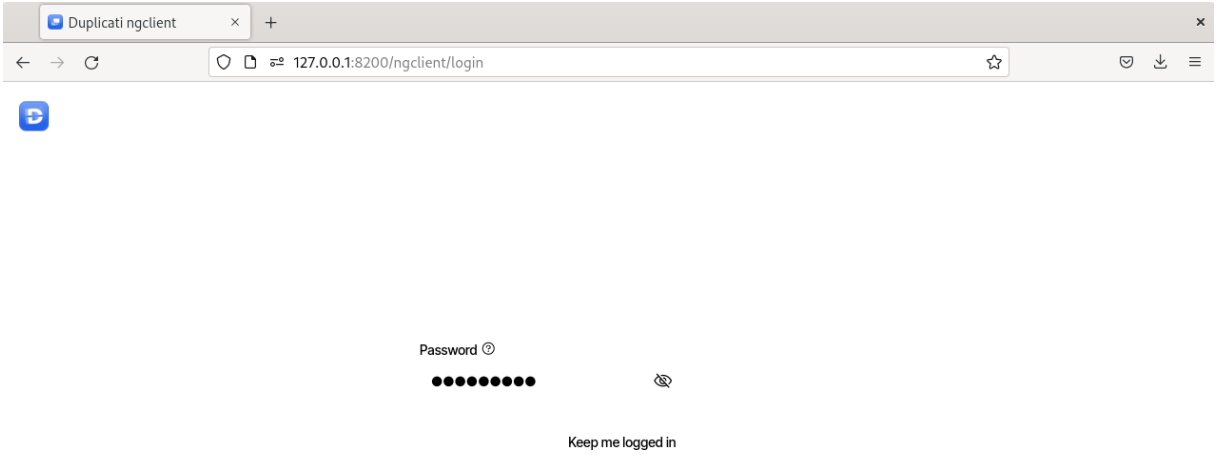
• Le dossier de stockage /home/test1/duplicati_sauvegarde



On configure le dossier qu'on veut sauvegarder

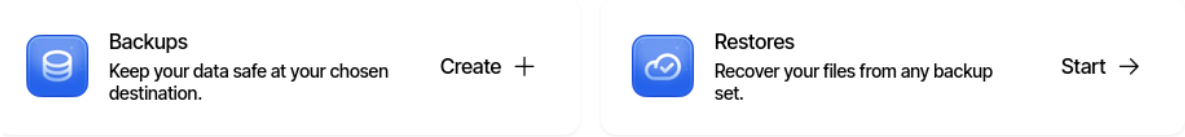


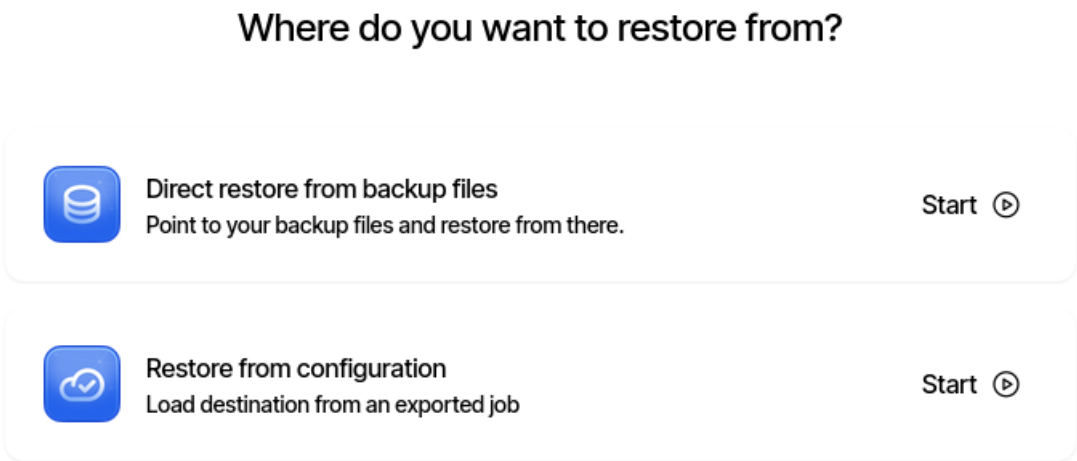
Sauvegarde automatique quotidiennement à 13h

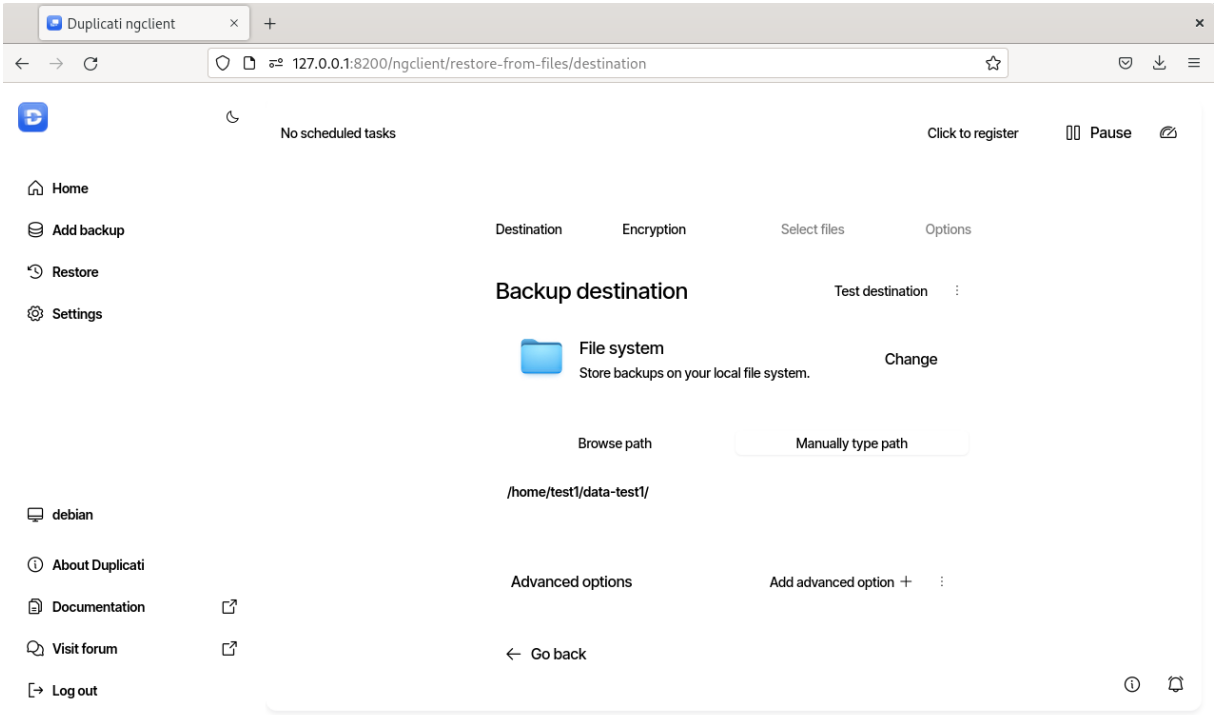


Duplicati chiffre les fichiers avec AES-256 (celui qu'on a choisi). Il demande donc le mot de passe pour accéder aux données de sauvegardes

2.3.3. Restauration







2.4. Comparaison des outils de sauvegardes

2.4.1. Fonctionnalités

Les outils de sauvegarde possèdent des fonctionnalités différentes, notamment dans la gestion de la déduplication et du chiffrement. Rsync et Rsnapshot sont limités à la copie de fichiers alors que les nouveaux outils modernes comme Restic, Borg et Duplicati transforment les données en blocs pour ne stocker que les modifications uniques, ce qui permet de réduire et d'optimiser l'espace disque.

Au niveau de la sécurité, Borg, Restic et Duplicati permettent le chiffrement (souvent AES-256) pour protéger les données avant même qu'elles soient envoyées.

Enfin, certains outils sont plus flexibles que d'autres pour la destination : Duplicati et Restic fonctionnent très bien avec les services Cloud (comme Google Drive), alors que Borg et Rsnapshot sont surtout utilisés pour des sauvegardes locales ou via SSH

2.4.2. Restauration à l'identique

Rsync et Rsnapshot sont très bons pour ça, car ils conservent l'arborescence originale sans la modifier, ce qui permet de vérifier visuellement. BorgBackup est plus particulièrement connu pour sa capacité à comparer les fichiers octet par octet avec

l'original, ce qui permet de vérifier que tout correspond parfaitement. Restic permet aussi de bien restaurer, mais ça peut être plus compliqué quand il y a beaucoup de fichiers. Duplicati est surtout adapté à la restauration de données personnelles (documents, photos, etc.). Il peut restaurer correctement, mais il n'est pas idéal pour restaurer un système complet avec toutes ses données

2.4.3. Performance

La performance d'un outil de sauvegarde dépend de sa vitesse de transfert de l'efficacité du stockage. BorgBackup est souvent le plus rapide en local, car il permet une bonne déduplication avec une compression. Restic est performant surtout pour les sauvegardes vers le Cloud, car il peut envoyer plusieurs blocs en parallèle. Duplicati est généralement plus lent car il repose sur le framework .NET, ce qui peut rallonger le temps de traitement lorsqu'il y a beaucoup de données. Rsync reste rapide pour transférer des fichiers, mais il devient moins efficace si les fichiers sont déplacés ou renommés, car il ne fait pas de déduplication comme les outils modernes.

3. Sécurité des sauvegardes

3.1. Problématiques

3.1.1. Confidentialité

Les sauvegardes contiennent des données sensibles comme des mots de passe, des documents personnels, une base de données ext... Si les données ne sont pas chiffrées, n'importe qui ayant accès aux sauvegardes peut lire l'intégralité des informations

3.1.2. Intégrité

Une sauvegarde doit être exacte et non altérée. En effet, les données peuvent être modifiées sans qu'on le sache ou corrompues avec le temps ce qui peut rendre les fichiers irrécupérables. Il est important que les données restaurées soient identiques à l'original

3.1.3. Disponibilité et ransomwares

Une sauvegarde doit être restaurable même en cas de panne du serveur principal, de perte de réseau, ou de suppression, qui peut arriver à cause des ransomwares. Ils ciblent aussi les sauvegardes et peuvent les chiffrer, les supprimer.

3.2. Solutions des outils de sauvegardes

3.2.1. Chiffrement

Les outils permettent le chiffrement du côté client. Les données sont chiffrées avant qu'on quitte notre machine. Donc si le stockage Cloud est piraté, les fichiers restent illisibles, cela permet de répondre à la problématique de confidentialité.

3.2.2. Déduplication

Les fichiers sont découpés en blocs. Si un ransomware essaie de modifier un fichier, cela ne touchera pas les blocs précédents déjà sauvegardés.

De plus, chaque bloc de données possède une empreinte numérique unique (le hachage) qui permet de vérifier qu'il n'a pas été modifié. Cela permet de répondre à la problématique d'intégrité

3.2.3. Immuabilité

Pour se protéger contre les ransomwares, on peut les empêcher de modifier la sauvegarde. Certains outils utilisent des fonctions de verrouillage (Object Lock) sur le stockage ce qui empêchent la suppression des fichiers pendant une période qu'on aura défini, même si on accède en tant qu'administrateur.